Infrastructure as Code

Serverless

Ron C. Chiang
University of St. Thomas



Agenda

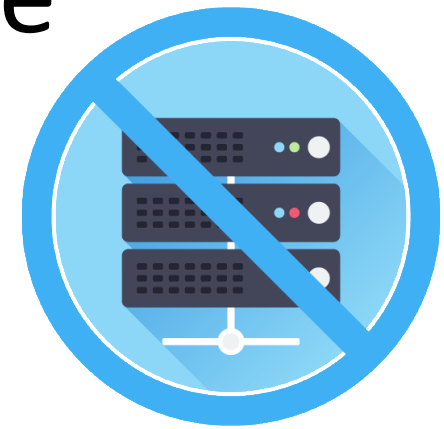
- Serverless and Lambda
  - How does it work?
  - Performance considerations
  - Programming model
  - Versioning
  - Best Practices
- API Gateway, DynamoDB, Kinesis, Cognito
- SAM & Serverless Frameworks

“There’s a lot of undifferentiated heavy lifting that stands between your idea and its success.”

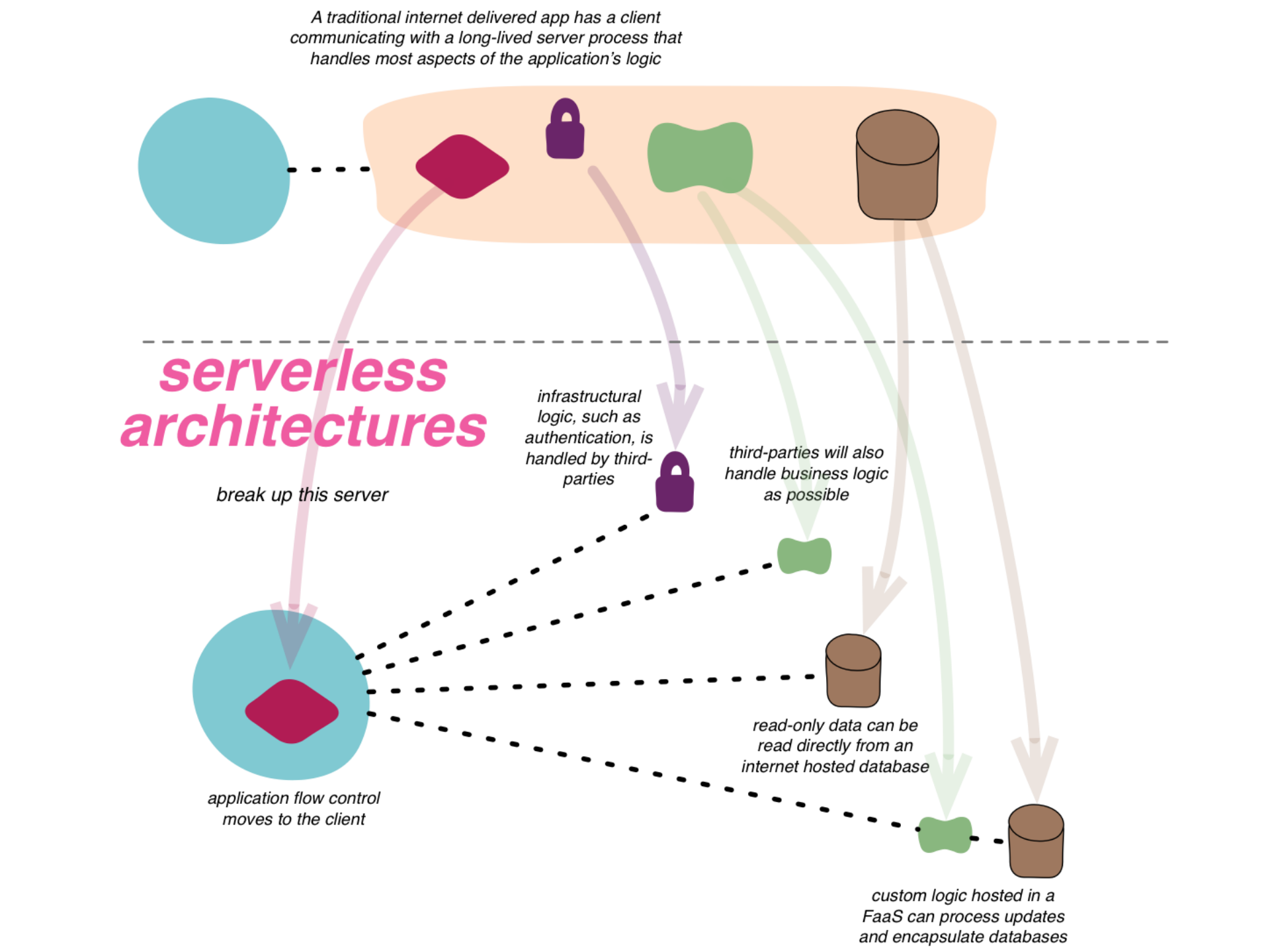
– *Jeff Bezos*



Serverless Architecture



- Cloud-based architecture leveraging an event-driven programming model primarily composed of managed resources.
  - We no longer manage compute infrastructure, operating systems, or application servers.
  - A Platform-as-a-Service (PaaS) model, but less opinionated than traditional solutions (like Elastic Beanstalk, Heroku).
  - We call it serverless, but our code has to run on servers somewhere.


A traditional internet delivered app has a client communicating with a long-lived server process that handles most aspects of the application's logic
serverless architectures
break up this server
infrastructural logic, such as authentication, is handled by third-parties
third-parties will also handle business logic as possible
application flow control moves to the client
read-only data can be read directly from an internet hosted database
custom logic hosted in a FaaS can process updates and encapsulate databases

AWS Lambda



A compute service which runs your code on-demand and scales automatically without the need to provision or manage servers.

- Key component of a serverless platform which runs our business logic.
- Only pay for the compute time you use in 100ms increments.
- Event-driven programming model
  - Code isn't running continuously, just when triggered by events.

Lambda use cases

- What kinds of things can we do with Lambda functions?
  - Build RESTful APIs and web applications
  - Build data analysis and transformation pipelines
  - Automate administrative tasks

How does it work?

- Let’s look at the steps involved in working with Lambda:

1. Write the application code
2. Create a Lambda function resource in AWS
3. Deploy the function
4. Run the function
5. Monitor the function

How does it work?

- **Write the Lambda function code**
  - Node.JS, Java, Python, C#, Go
  - Add code libraries
  - AWS SDK is built in and always available
  - Functions are stateless and ephemeral

```
1  console.log('Loading function');
2  var aws = require('aws-sdk');
3  var s3 = new aws.S3({ apiVersion: '2006-03-01' });
4  exports.handler = function(event, context) {
5      var bucket = event.Records[0].s3.bucket.name;
6      var key = decodeURIComponent(event.Records[0].s3.object.key.replace(/\+/g, ' '));
7      var params = {
8          Bucket: bucket,
9          Key: key
10     };
11     s3.getObject(params, function(err, data) {
12         if (err) {
13             console.log(err);
14             var message = "Error getting object " + key + " from bucket " + bucket;
15             console.log(message);
16             context.fail(message);
17         } else {
18             console.log('CONTENT TYPE:', data.ContentType);
19             context.succeed(data.ContentType);
20         }
21     });
22 };
```

How does it work?

- **Create a Lambda function resource**
  - Create via AWS Lambda web console, CloudFormation, AWS API
  - Need to specify:
    - Region function will run in
    - Amount of memory to allocate from 128MB to 3008MB in 64MB increments
      - CPU and network are proportional to allocated memory
    - Execution role function will use to access AWS resources
    - Concurrency level: number of simultaneous Lambda function executions allowed

How does it work?



- **Deploy your code**
  - Author Lambda code directly on the AWS Console
  - Package code and libraries into a zip file and upload to Lambda
    - Can upload to S3 and point Lambda function at S3 package
  - Create CloudFormation template with SAM extension or Serverless framework
  - Publish function from IDE
    - Microsoft Visual Studio
    - AWS Cloud9

How does it work?

- **Run your code**
  - Event-driven trigger from AWS resources (event source mapping)
    - S3, DynamoDB, Kinesis, SNS, CloudWatch, and more
  - Requests received from an AWS endpoint:
    - AWS API Gateway – HTTP
    - AWS IoT – MQTT
  - Scheduled trigger (CloudWatch/ Cron)

How does it work?

- Monitor your code
  - Code metrics instrumented by CloudWatch
    - Request count, Error rate, Latency, Throttling
  - Execution logs stored in CloudWatch logs
    - Application logs and stack traces
  - AWS X-ray: instrument code to provide distributed application tracing
    - Troubleshoot function latency and long-running queries



X-ray trace


Clients
avg. 625ms
18 t/min
selfies-processor-demo
AWS::Lambda
avg. 26.3s
34 t/min
selfies-processor-demo
AWS::Lambda::Function
avg. 39.6s
19 t/min
selfies-table2
AWS::DynamoDB::Table
avg. 2.28s
34 t/min
AmazonRekognition
AWS::AmazonRekognition

How does it work?

- How is AWS actually running your code?
  - Event trigger initiates creation of Lambda execution environment using a container (cold start).
  - Lambda deploys your code into the environment and provides any bootstrapping (like starting up JVM).
  - Code executes like any normal application code.
    - A single event runs per container.
    - No need to worry about blocking operations.
  - The environment may persist for some period of time and subsequent events may trigger code execution in the same environment (warm start).
    - Environment contains a /tmp directory with 500MB of storage space.



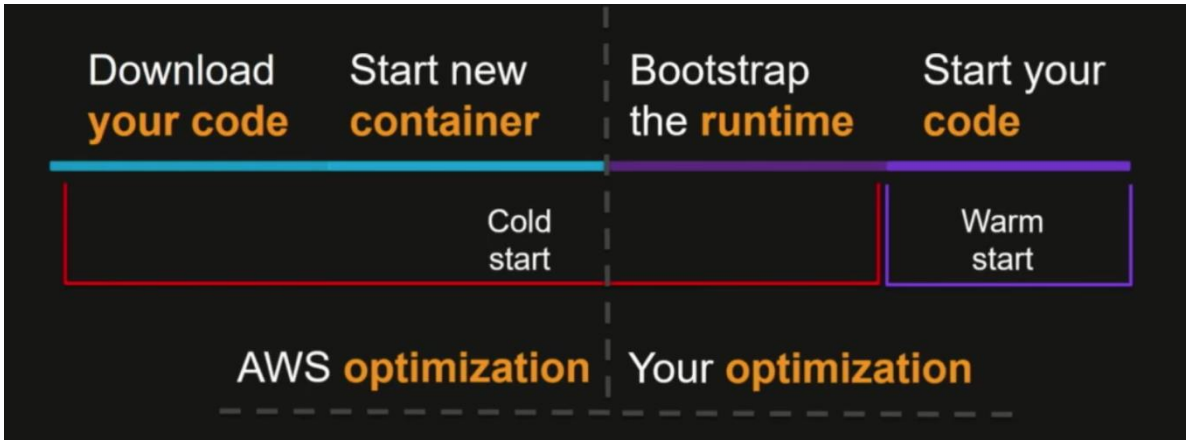
How does it work?

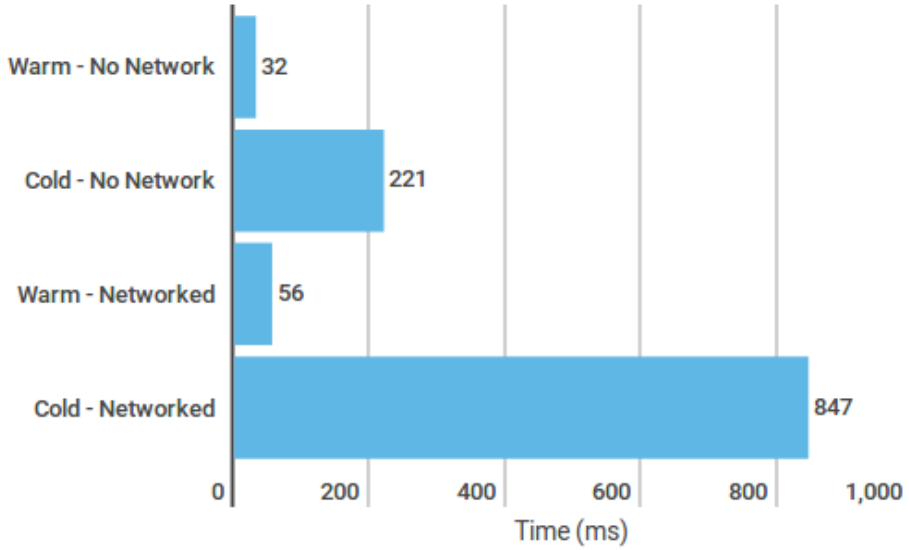
- Lambda pricing model:
  - First 1 million requests per month are FREE
    - $0.20 per 1M requests thereafter
  - 400,000 GB-Seconds of compute time per month FREE
    - $0.00001667 per GB-Second thereafter
    - Notice that pricing is a function of execution time * memory allocation
    - 128MB function running for 8 seconds = 1 GB-Second
  - A Lambda which connects to other services like S3 may incur additional charges

Lambda function performance

- Cold vs. Warm starts.
  - Lambda execution environment may remain available for some period of time.
    - There's no guarantee this will be the case!
  - Frequently exercised Lambda functions will generally exhibit lower latency.
  - Cold start time can greatly vary based on the nature of the function.
    - Framework type – Java (slower) vs. Python (faster)
    - Bootstrapping connections to AWS resources (DynamoDB)
    - Connecting to VPC-based resources (RDS)

Lambda function performance





Lambda Programming Model

- Common components which apply to all programming languages:
  - **Handler**: the function which Lambda calls to start the execution of your code
  - **Event**: an object which is passed to the handler containing event source information and parameter values
  - **Context**: an object which contains Lambda run-time information
  - **Callback**: an optional object used in async platforms like Node.JS to return values to the caller
  - **Return**: the handler can optionally return a value

Lambda Programming Model

- Handler component: function invoked by Lambda when it runs your code
  - Think of this as the main function
  - Specified in the Lambda configuration using *my_function_filename.handler_name* format

```
def handler_name(event, context):
    ...
        return some_value
```

- Python example:

```
def my_handler( event, context):
    message = 'Hello {}'.format(event['first_name'])
    return { 'message' : message }
```

Lambda Programming Model

- Event object: passed to the handler by the event source
  - The format of the event object varies based on the event source
  - https://docs.aws.amazon.com/lambda/latest/dg/eventsources.html

- Example (CloudFormation create request):

```
{
"StackId": "arn:aws:cloudformation:us-west-2:EXAMPLE/stack-name/guid",
"ResponseURL": "http://pre-signed-S3-url-for-response",
"ResourceProperties": {
    "StackName": "stack-name",
    "List": [ "1", "2", "3" ] },
"RequestType": "Create",
"ResourceType": "Custom::TestResource",
"RequestId": "unique id for this create request",
"LogicalResourceId": "MyTestResource"
}
```

Lambda Programming Model

- Context object: allows you to interact with the Lambda service to get runtime information
  - How much time is remaining before Lambda terminates the running function?
  - Which CloudWatch log group is associated with the running function?

- Methods and attribute names depend on programming platform
  - context.function_name: name of the Lambda function
  - Identity.function_name: information about the Cognito identity provider

- Example:

```
def remaining_time(context):
  return context.get_remaining_time_in_millis()
```

Lambda Programming Model

- Logging
  - Stdout/stderr messages are written to CloudWatch logs for each function execution
  - Use a platform-dependent log library for more detailed logs including error level and timestamps (i.e., import logging)

- Where to find logs:
  - Log section of the Lambda console
  - CloudWatch logs console (need to know specific log group name and log stream)
  - Pass in the LogType parameter when invoking a function (X-Amz-Log-Type: *Tail*) and the function will return last 4k of log data in x-amz-log-result header

Testing Lambdas

- AWS Lambda web console
  - Create and save test events
- Deploy to AWS and manually trigger
  - Access API Gateway endpoint with Postman
  - Generate event (create S3 object, SQS message, etc.)
- Unit testing with fixtures/ mocking
  - SAM CLI (https://github.com/awslabs/aws-sam-cli)
  - Placebo (https://github.com/garnaat/placebo)

Hands-on Demo

- Create a basic Hello World Lambda function

Lambda environment variables

- Environment variables are key-value pairs which are dynamically passed to Lambda functions during execution.
  - Decouple application settings from function logic
  - Different sets of variables for dev, staging, prod
- No limitation on number of variables, but total set must be < 4 KB.

Lambda function versioning

- Lambda functions may be versioned.
  - Versioning creates a snapshot of both the function code and the configuration
  - Qualified function reference:
    - arn:aws:lambda:aws-region:acct-id:function:helloworld**:$LATEST**
  - Unqualified function reference:
    - arn:aws:lambda:aws-region:acct-id:function:helloworld
  - Versions may be specified during creation
    - Function versions are immutable (including config like env vars)
    - arn:aws:lambda:aws-region:acct-id:function:helloworld:2
  - Allows us to provide support for multiple API versions

Lambda execution



- AWS imposes default limits on Lambda execution:
  - Maximum execution duration: 900 (15 minutes)
  - Request body payload (synchronous): 6MB
  - Request body payload (async): 128KB!
    - Avoid things like images in payload.
  - Number of file descriptors/ processes/ threads: 1,024
  - Concurrent executions per region: 1,000
  - Function deployment package size: 50MB compressed/ 250MB uncompressed
  - Total size of all function packages in a region: 75GB

Accessing VPC Resources

- A Lambda function cannot connect to VPC resources by default.
  - It runs in a VPC, just not a private VPC owned by you.
  - Examples of VPC resources: EC2, RDS, ElastiCache instances



- Need to configure Lambda function to connect to a VPC
  - Specify a VPC and Security group when creating/updating function
  - AWS Lambda creates an Elastic Network Interface (ENI) for function which provides a private IP address within VPC for communication.

- Issues:
  - Each concurrently running Lambda function instance requires an ENI/private IP
  - Need to make sure VPC can provide enough private IP's to support potential usage levels
  - It may take up to 10s to connect an ENI to a VPC!

Best Practices

- Separate Lambda handler function from core logic to make unit testing easier.

- Take advantage of execution context to improve performance:
  - Use static constructors, global/static variables and singletons.
  - Keep alive and reuse database connections established during previous invocation.

```
import boto3
client = None

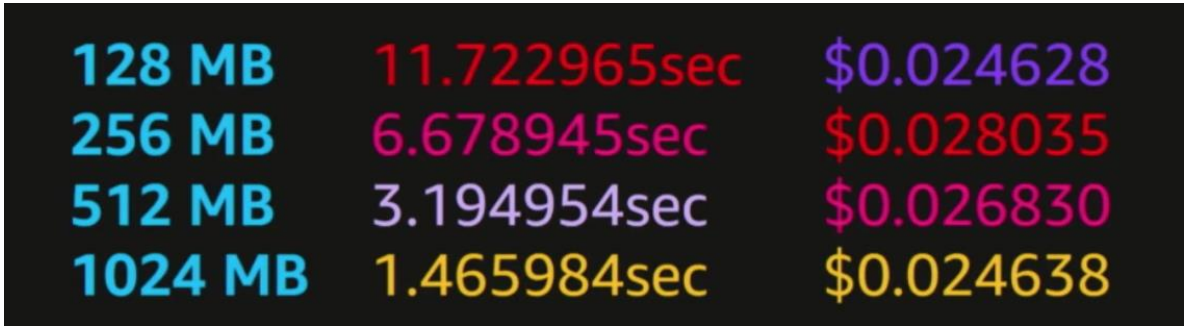
def my_handler(event, context):
    global client
    if not client:
            client = boto3.client("s3")
…
```

Best Practices

- Be careful about using pre-existing libraries in Lambda environment (i.e., boto3) because AWS may change these at any time
  - Package required libraries with your function code.
- Use less complex frameworks that load faster.
- Avoid using recursive code.
- Don’t wait for I/O in your function, leverage an external orchestration tool (example: Step Functions).
- Match function concurrency to database scalability: don’t run 300 concurrent functions against DB that only supports 100 open connections!

Best Practices

- Use the most restrictive permissions when specifying Lambda execution policies.
- Load test functions to determine optimal timeout value: avoid unexpected concurrency spikes.
- Functions with large memory footprint are not necessarily more expensive than smaller memory footprint!
  - (https://github.com/alexcasalboni/aws-lambda-power-tuning)



API Gateway



- An AWS service which translates web requests into Lambda events
  - Trigger Lambdas via a web browser client.
  - Communicate directly with DynamoDB.
  - Scales automatically based on demand.

- Takes on the role of the front-end webserver in a traditional application stack
  - Allows us to build a RESTful API

- AWS deploys the API Gateway in a CloudFront distribution.
  - Regional or Edge-optimized

- Supports API caching to increase performance.

API Gateway

- Key API components:
  - Resources: These are the nouns in a RESTful API
    - https://myApi.com/api/**user**
    - https://myApi.com/api/**company**
    - https://myApi.com/api/**order**
  - Methods: These are the verbs in a RESTful API
    - HTTP **GET**
    - HTTP **POST**

- We combine resources and methods to create an API:
  - GET https://myAPI.com/api/user?id=123

- API resources can be defined manually or using Swagger

DynamoDB



- A NoSQL AWS database that supports key-value and document (JSON) storage

- Key components:
  - **Tables**: a way to group/partition data
  - **Items**: a grouping of data attributes in a table (kind of like a row in a relational table)

- Tables contain sets of items and items contain sets of attributes.
  - Unlike a relational database, each item can contain different attributes.

DynamoDB

- Each table must have a unique primary key
  - Two types:
    - **Partition/hash key**: a single attribute which is used as a hash to determine where to store the data
      - userId, companyId, orderId, etc
    - **Partition key + sort key** (composite key): the partition key determines where to store the data and the sort key determines the ordering of data
      - Sort key = date/time, counter, etc.
      - Items could share partition key but must have unique sort key

DynamoDB

- You perform queries on DynamoDB using the primary key

```
{
    TableName: "Music",
    KeyConditionExpression: "Artist = :a and SongTitle = :t",
    ExpressionAttributeValues: {
        ":a": "No One You Know",
        ":t": "Call Me Today"
    }
}
```

In SQL syntax this would look like:

```
SELECT * FROM Music WHERE Artist='No One You Know' AND SongTitle = 'Call Me Today';
```

DynamoDB

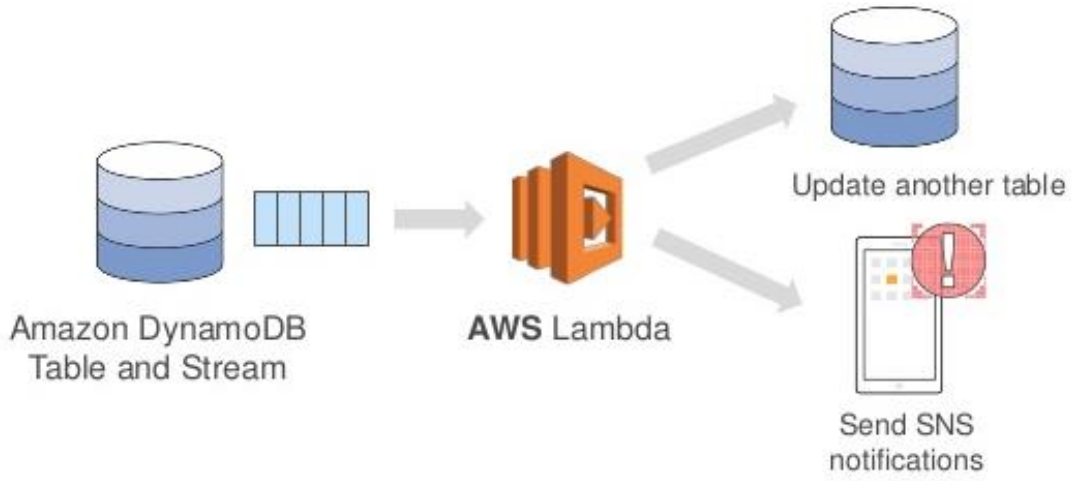
Query response:


```
{       "Count": 2,
        "Items": [
                {
                        "Artist": { "S": "Big Rock Band" },
                        "SongTitle": { "S": "Cool Song Name" },
                        "ReleaseDate": { "S": "20150422" }
                },
                {
                        "Artist": { "S": "Sizzling Singers" },
                        "SongTitle": { "S": "Just another love song" },
                        "ReleaseDate": { "S": "19760202" }
                },

        ],
        "ScannedCount": 2,
        "ConsumedCapacity": null
}
```

DynamoDB

- **Streams**: DynamoDB events can be streamed (event source) to lambda functions



  - Allows you to emulate functionality similar to stored procedures in a relational database

Kinesis

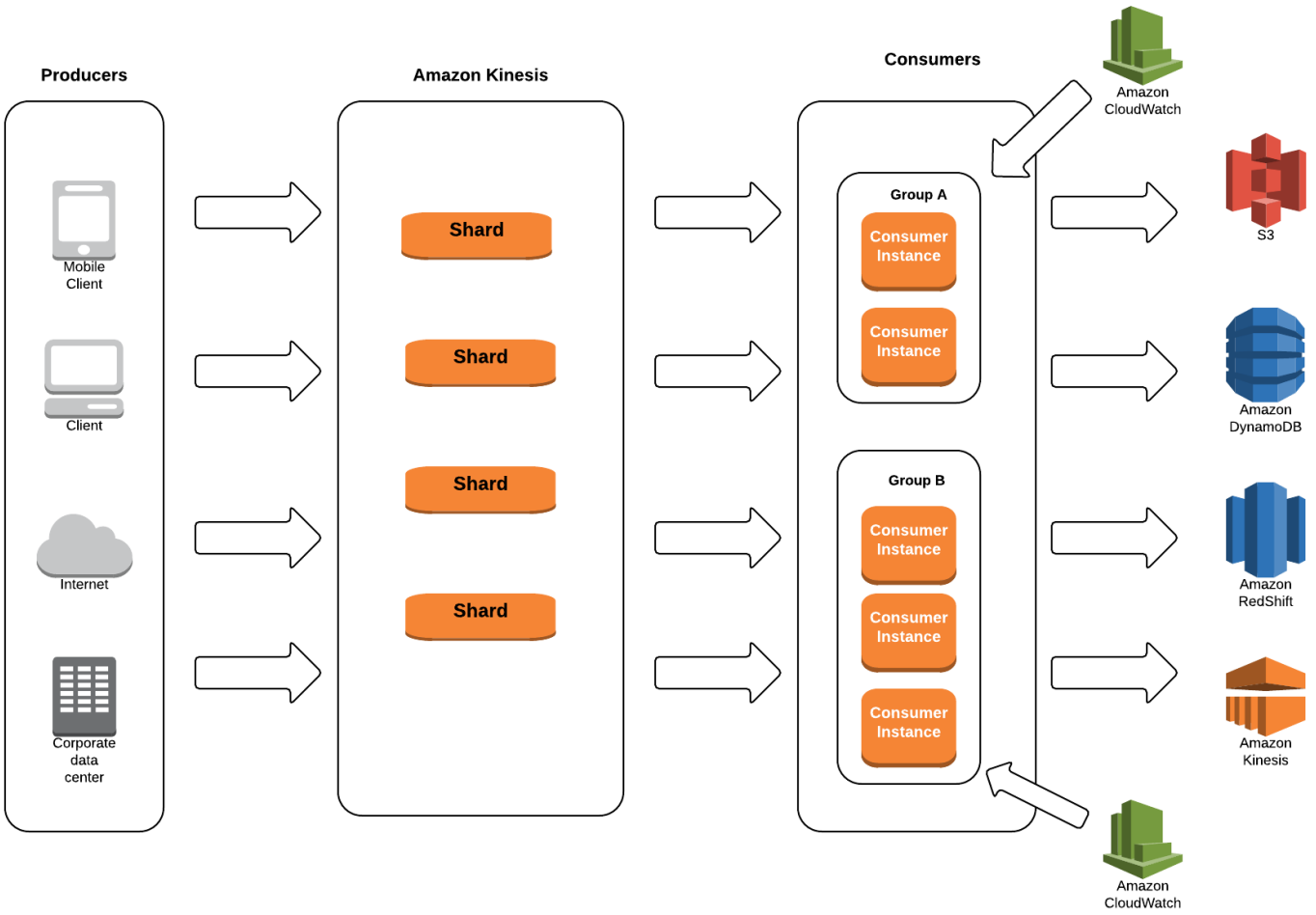


- Kinesis is designed to ingest and analyze massive streams of data.
  - Real-time orders from thousands of stores.
  - Event messages from millions of IoT devices.

- Three different types:
  - Kinesis Streams
  - Kinesis Firehose
  - Kinesis Analytics

- **Kinesis Streams** are massively scaled message queues
  - Incoming messages stored in shards based on a partition key
  - Messages stored for at least 24 hours and up to 7 days.
  - Multiple consumers can read messages in shards
    - Shard reads = max 5 TPS & 2MB/s
    - Shard writes = max 1000 records/s & 1 MB/s

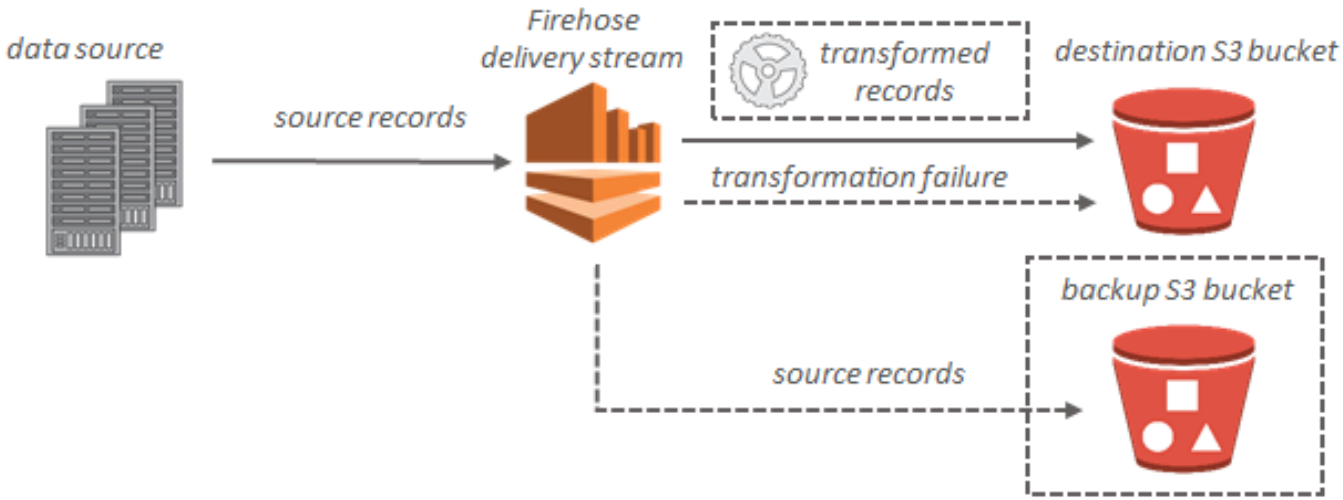
Kinesis Streams



Kinesis Firehose

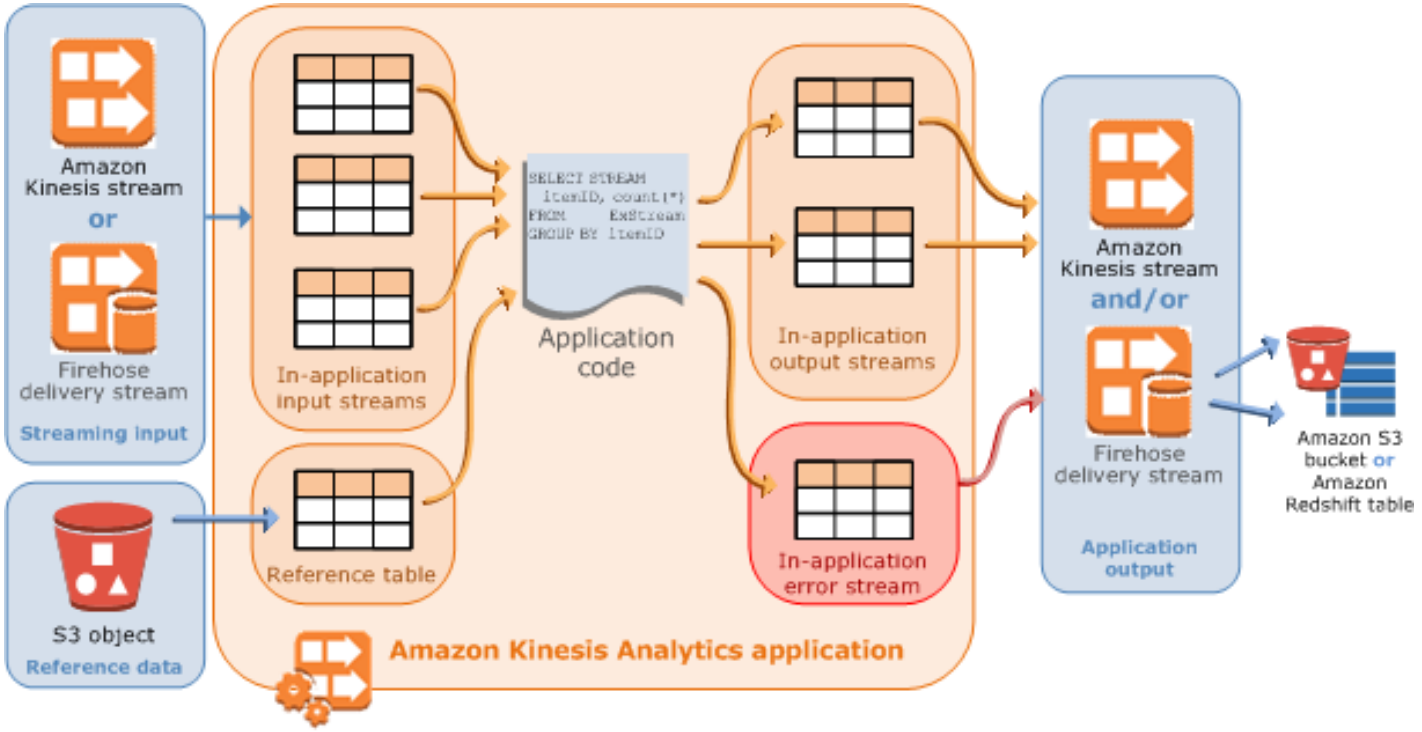


- **Kinesis Firehose** is used to transform data and store it in a persistent location.
  - Ingested data may be transformed by a lambda function (optional).
  - Data is not persisted in the stream and must be stored somewhere like S3 or ElasticSearch.



Kinesis Analytics

- **Kinesis Analytics** works with Kinesis Streams and Kinesis Firehose to analyze data in real-time and store this data in S3, Redshift, or ElasticSearch.



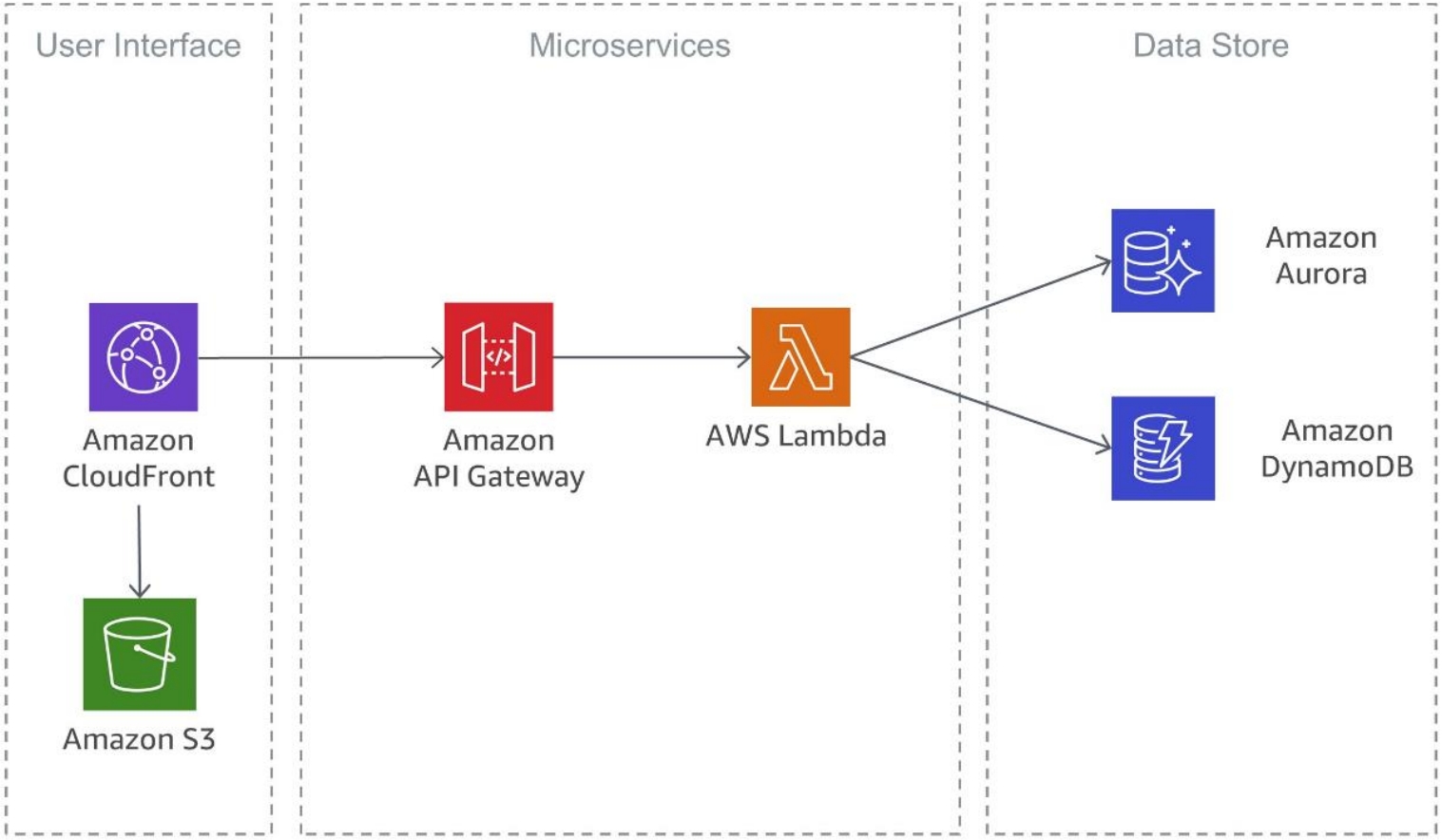
Cognito



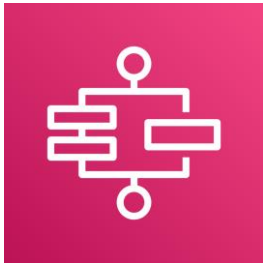
- A managed Identity Management Service which supports user account management.
  - Manage user accounts for your application.
  - Create sign-up and login pages for users.
  - Federate logins with corporate Active Directory systems or other vendors like Facebook.

- Cognito will exchange access tokens (JWTs) for temporary AWS credentials.

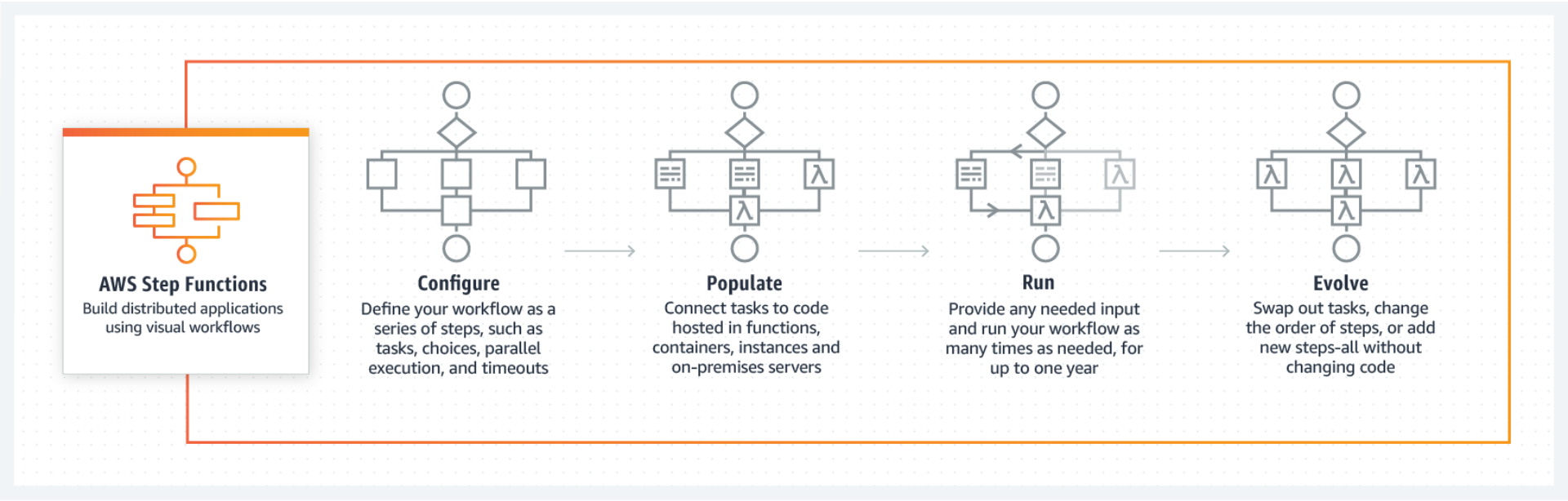
Putting it all together



Step Functions



- Leverage lambdas and a state-machine to create workflows.



AWS SAM

- CloudFormation was originally designed for AWS network, compute and storage resources.

- AWS Serverless Application Model (SAM) introduces a set of extensions for serverless applications:
  - Lambda functions
  - API Gateway
  - DynamoDB



AWS SAM Template

- SAM template is designated by an additional Transform section:

```
AWSTemplateFormatVersion: '2010-09-09’
Transform: 'AWS::Serverless-2016-10-31’
Resources:
    MyFunction:
        Type: 'AWS::Serverless::Function’
        Properties:
            Handler: index.handler
            Runtime: nodejs6.10
            CodeUri: 's3://my-bucket/function.zip'
```

SAM Globals

- The Globals section allows you to define properties common to all functions and APIs.

```
Globals:
    Function:
        Runtime: nodejs6.10
        Timeout: 180
        Handler: index.handler
        Environment:
            Variables: TABLE_NAME: data-table
    Api:
        EndpointConfiguration: REGIONAL
        Cors: "'www.example.com'"
```

Serverless Function

- Resource type: **AWS::Serverless::Function**
  - Creates Lambda function, IAM execution role, and event source mappings

```
Resources:
  MyLambdaFunction:
    Type: AWS::Serverless::Function
    Properties:
      ...
      MemorySize: 1024
      ...
```

Serverless Function

```
Handler: index.js
Runtime: nodejs6.10
CodeUri: 's3://my-code-bucket/my-function.zip'
Description: Creates thumbnails of uploaded images
MemorySize: 1024
Timeout: 15
Policies:
 - AWSLambdaExecute # Managed Policy
 - Version: '2012-10-17' # Policy Document
   Statement:
    - Effect: Allow
      Action:
       - s3:GetObject
       - s3:GetObjectACL
      Resource: 'arn:aws:s3:::my-bucket/*'
Events:
  PhotoUpload:
    Type: S3
    Properties:
     Bucket: my-photo-bucket
```

Serverless Function

- Event source types:
  - S3
  - SNS
  - Kinesis
  - DynamoDB
  - Api
  - Schedule
  - CloudWatchEvent
  - CloudWatchLogs
  - IoTRule
  - AlexaSkill

```
Type: S3
Properties:
  Bucket: my-photo-bucket
  Events: s3:ObjectCreated:*
```

```
Type: Api
Properties:
  Path: /photos
  Method: post
```

Serverless Function

- Lambda code can live alongside the CloudFormation template

```
/project-directory
        /cf-template.yaml
        /myfunction
                /index.py


Resources:
        MyFunction:
                Type: 'AWS::Serverless::Function’
                Properties:
                        Handler: index.handler
                        Runtime: python3.7
                        CodeUri: ./myfunction
```

Serverless Database

- Resource type: **AWS::Serverless::SimpleTable**
  - Creates DynamoDB table with a single attribute primary key
  - Use AWS::DynamoDB::Table resource for more advanced DynamoDB configurations

```
Resources:
  MyTable:
    Type: AWS::Serverless::SimpleTable
    Properties:
      TableName: my-table
    PrimaryKey:
      Name: id
      Type: String
    ProvisionedThroughput:
      ReadCapacityUnits: 5
      WriteCapacityUnits: 5
    Tags:
      Department: Engineering
      AppType: Serverless
```

CloudFormation SAM

- Two commands to package and deploy CloudFormation SAM:
  - aws cloudformation package
  - aws cloudformation deploy